Challenge 2 — Kernel Methods for Classification and Dimensionality Reduction

Overview

The goal of this challenge was to investigate when kernelized methods outperform their linear counterparts on two pre-built toy datasets. Specifically, I explored **Support Vector Machines (SVMs)** with different kernels for classification, and **Kernel PCA** vs. standard PCA for dimensionality reduction and class separability. All experiments were implemented in Python using scikit-learn, with systematic hyperparameter search and visual inspection of results.

Part 1 — Linear SVM vs. Kernel SVM

Dataset

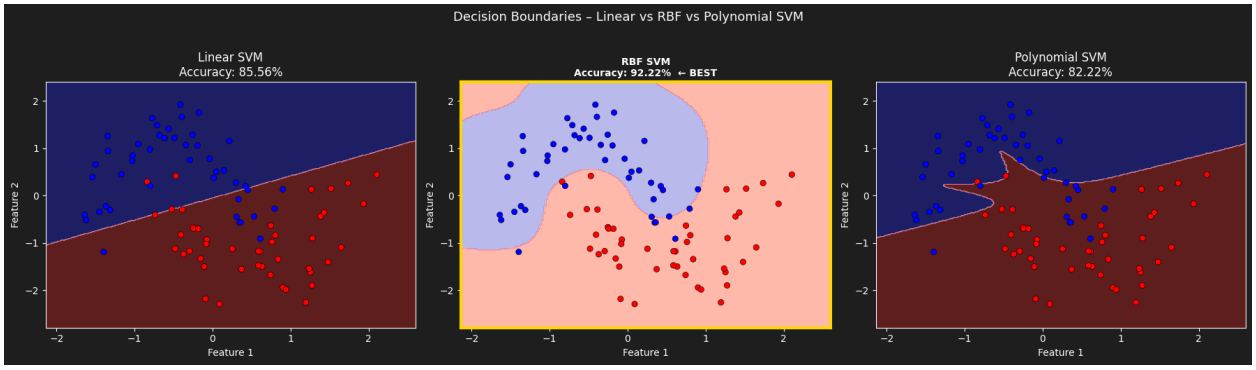
The dataset used for this task was a pre-generated `make_moons` dataset (300 samples, noise=0.2) — an intentionally non-linearly separable dataset where the two classes form crescent-moon shapes that interleave in feature space. Features were standardized with `StandardScaler` before training, as SVMs are sensitive to feature scale.

What I Did

I trained three SVM variants on the provided dataset:

- **Linear SVM** — a baseline with a linear decision boundary
- **RBF SVM** — using a Radial Basis Function (Gaussian) kernel, with `C` and `gamma` tuned via 5-fold cross-validated `GridSearchCV`
- **Polynomial SVM** — using a polynomial kernel, with `C` and `degree` explored via grid search

Results & Analysis



The Linear SVM struggled as expected — 86% accuracy reflects the fundamental inability of a linear hyperplane to cleanly separate crescent-shaped classes. The **RBF kernel was the clear winner at 92%**, demonstrating that the Gaussian kernel can effectively map data into a higher-dimensional space where the classes become linearly separable. Looking at the classification report, the RBF SVM achieved strong precision and recall for both classes (macro avg F1 ≈ 0.92), while the linear model showed a visible class imbalance in performance (F1 of 0.84 vs 0.87 across classes).

The polynomial SVM underperformed at 82%, with notably poor recall on class 0. This suggests that for this particular data geometry, polynomial kernels — even with the best hyperparameters found — cannot capture the curvature as efficiently as the RBF kernel. The best polynomial configuration found was degree=5 with gamma=5.0, indicating that the grid search pushed toward high-complexity configurations to compensate for the kernel's structural mismatch with the data.

Key takeaway: For non-linearly separable data with curved, interleaved boundaries, the **RBF kernel is the most appropriate choice** due to its ability to model smooth, locally-adaptive decision surfaces.

Part 2 — PCA vs. Kernel PCA on `make_moons`

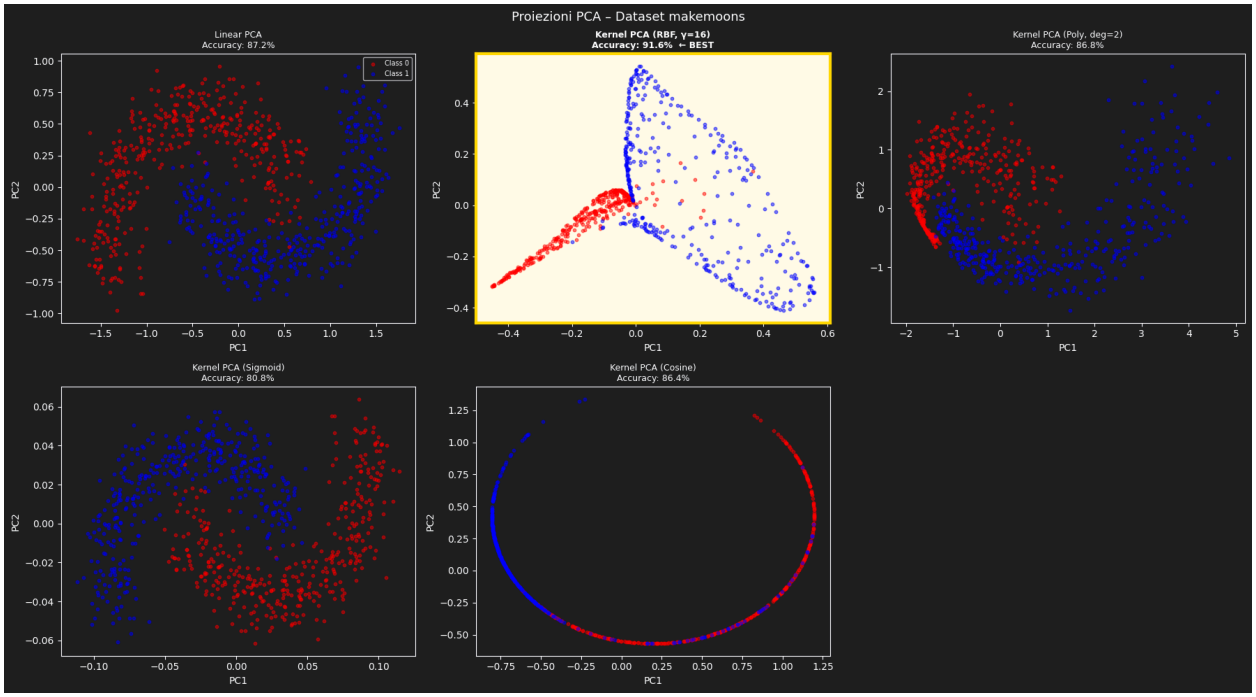
Dataset

A pre-generated moon dataset (1,000 samples, noise=0.15) was provided for this task. The same non-linear crescent structure applies, making it a meaningful benchmark for comparing linear vs. non-linear dimensionality reduction.

What I Did

I applied standard PCA and Kernel PCA to the provided dataset, projecting the data onto 2 components. The quality of each projection was evaluated by training a **Logistic Regression** classifier on top of it. Kernel PCA was explored with multiple kernels (RBF, polynomial, sigmoid, cosine, linear) through a `GridSearchCV` pipeline.

Results & Analysis



Standard linear PCA failed to separate the two moon classes — the result is two overlapping clusters in the projected 2D space, as PCA can only find linear projections. Kernel PCA with an **RBF kernel dramatically improved separability**, implicitly mapping the data into a space where the two moons become linearly separable. The Logistic Regression accuracy on the Kernel PCA projection was substantially higher than on the standard PCA projection, confirming that the kernel trick enables meaningful non-linear feature extraction.

Key takeaway: When the underlying data structure is non-linear, standard PCA is fundamentally limited, while Kernel PCA — especially with an RBF kernel — can reveal latent class structure that is invisible to linear methods.

A Note on gamma=16 in Kernel PCA (make_moons): I've selected a relatively high value of **gamma=16** for the RBF kernel in Kernel PCA. At first glance, this may seem counterintuitive — high gamma values make the Gaussian kernel very narrow, meaning each training point influences only a very small neighbourhood around itself. However, in the context of the moon dataset, this actually makes sense.

The two crescent shapes are **tightly interlocked with a fine-grained boundary** between them. A low gamma would produce a broad, smooth kernel that blurs the distinction between the two classes, effectively "averaging out" the local structure that separates them. A high gamma, on the other hand, forces the kernel to be sensitive to **local geometry**, capturing the precise curvature and tight separation between the two moons.

In other words, gamma=16 works well here precisely because the relevant structure in the data is **local, not global** — the classes are not separated by a wide margin, but by a narrow, curved boundary that requires a fine-grained kernel to detect. This is consistent with the broader principle that gamma acts as an inverse length scale: higher gamma → shorter range → more sensitivity to local patterns.

It is worth noting that such a high gamma would be a red flag in many other settings, as it risks overfitting by making the model overly sensitive to individual data points. In this case, the regularisation parameter `C` likely plays a compensating role by controlling the margin hardness.

Part 3 — PCA vs. Kernel PCA on `make_blobs` with Linear Transform

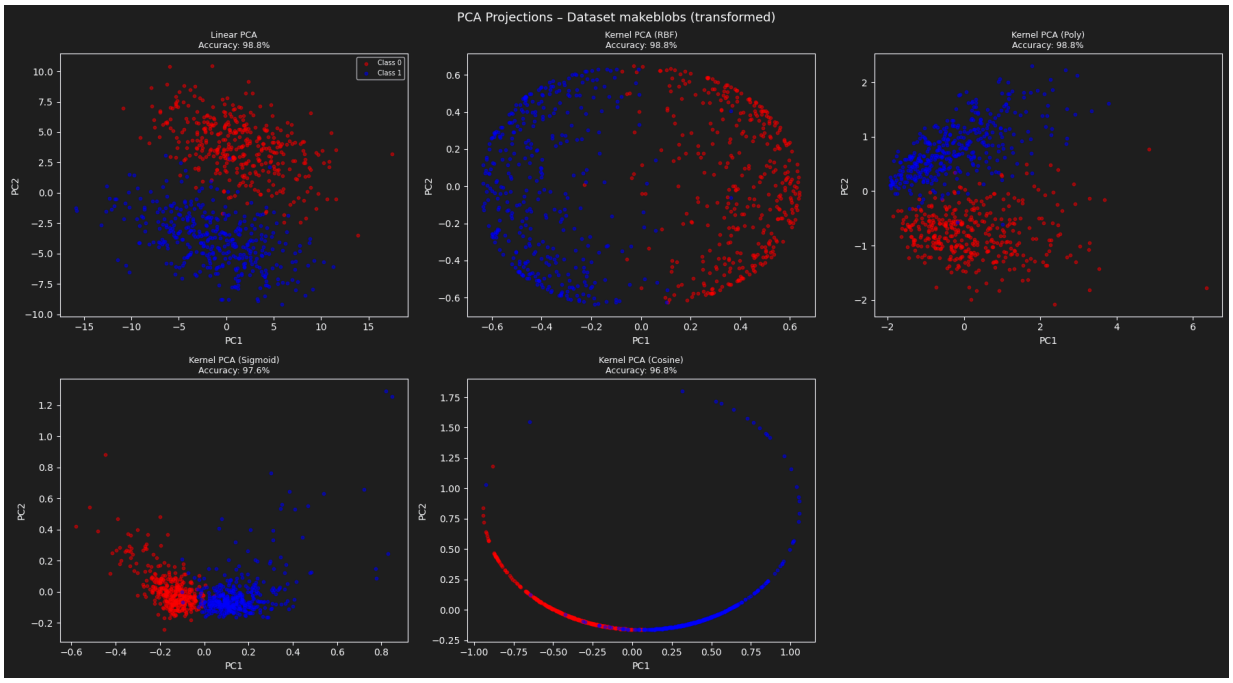
Dataset

For this task, a pre-generated and pre-transformed `make_blobs` dataset was provided. The linear transformation applied to the blobs preserves the intrinsically linear separability of the data, making this a deliberate contrast to Part 2.

What I Did

I repeated the PCA vs. Kernel PCA comparison on this dataset, again using a Logistic Regression classifier to evaluate the quality of the learned representations.

Results & Analysis



With a **linear kernel**, Kernel PCA achieved **96.80% cross-validation accuracy and 98.80% test accuracy**. Standard PCA performed comparably. This is expected: when classes are linearly separable, the kernel trick offers no advantage — a linear PCA is already optimal. In fact, Kernel PCA with a linear kernel is mathematically equivalent to standard PCA in this regime.

This experiment reinforces a fundamental principle in machine learning: **model complexity should match data complexity**. Introducing a non-linear kernel on intrinsically linear data adds unnecessary computational overhead with no benefit.

General Conclusions

The challenge demonstrated that — the **inductive bias of a model must align with the structure of the data**:

- **Non-linear data (moons)** → kernel methods (RBF SVM, RBF Kernel PCA) significantly outperform linear alternatives
- **Linearly structured data (blobs)** → linear methods are sufficient and computationally cheaper
- **Hyperparameter tuning via grid search** is essential for kernel methods, since parameters like `C`, `gamma`, and `degree` drastically affect decision boundaries and generalization performance